

Redes neuronales como aproximadores numéricos para pricing de opciones bajo Black-Scholes y Heston

Ángel Fernández Sánchez Jorge Alfageme Sotillos Héctor Pérez Ledesma

Proyecto de investigación — Métodos numéricos

Resumen

Este trabajo estudia bajo qué condiciones una red neuronal profunda puede actuar como un *surrogate* preciso, rápido y diferenciable de una función de pricing de opciones, entendida como sustituto numérico de un solver matemático ya conocido. La motivación no es predecir nada sobre el mercado sino abaratar la evaluación de modelos cuya solución analítica no existe o es cara: una vez entrenada offline a partir de datos sintéticos generados por el propio solver, la red puede sustituir al modelo original en tareas que exigen millones de evaluaciones, como la calibración diaria, el cálculo de Greeks o la simulación masiva de escenarios. Tomamos como referencia central el trabajo de Chen, Didisheim y Scheidegger sobre *deep surrogates* [4] y como complemento metodológico el de Huge y Savine sobre *differential machine learning* [5]. Nuestro objeto de estudio son las opciones europeas vainilla bajo dos modelos, Black-Scholes como caso de validación con solución cerrada y Heston como caso principal, donde la integración por Fourier semi-cerrada es lo suficientemente costosa como para justificar el surrogate. El trabajo se articula en torno a cinco experimentos comparativos que varían cada uno una sola dimensión del problema: la elección de métrica de evaluación, la activación de la red, la distribución del muestreo, la eficiencia computacional y el uso de información diferencial en la pérdida. Este documento describe la revisión bibliográfica que sustenta el planteamiento, la metodología experimental, el diseño concreto de cada comparación y la estructura del código que la implementa.

Palabras clave: *deep surrogates*, pricing de opciones, Heston, Black-Scholes, *differential machine learning*, métodos numéricos.

Índice

1. Introducción	2
2. Revisión bibliográfica	3
2.1. Hutchinson, Lo y Poggio (1994): el punto de partida histórico	3
2.2. Becker, Cheridito y Jentzen (2019): deep optimal stopping	4
2.3. Ruf y Wang (2020): mapa del campo	4
2.4. Chen, Didisheim y Scheidegger (2025): referencia central	5
2.5. Huge y Savine (2020): differential machine learning	6
2.6. Otras líneas complementarias	7
3. Enfoque del trabajo	8
4. Metodología	9
4.1. Pipeline experimental	9
4.2. Targets, normalización y métricas	10
4.3. Solvers y validación numérica	10
4.4. Dominio y muestreo	11

4.5. Particiones y evaluación por bins	12
4.6. Constantes del pipeline	13
5. Diseño experimental	14
5.1. Once surrogates	15
5.2. Criterios de validez	16
6. Plan de ejecución por fases	16
7. Discusión y trabajo futuro	17

1. Introducción

El objetivo general de este trabajo es estudiar cómo se pueden usar las redes neuronales como herramienta numérica para aproximar modelos de pricing de opciones. Es decir, no nos interesa la red como un sistema que aprende del mercado, sino como un aproximador funcional capaz de imitar a un solver matemático ya conocido. La distinción puede parecer sutil pero cambia completamente lo que se está haciendo: no estamos prediciendo nada, estamos sustituyendo un cálculo costoso por una aproximación rápida y diferenciable.

En finanzas cuantitativas hay muchos modelos de pricing de opciones, desde el clásico Black-Scholes hasta modelos modernos con volatilidad estocástica, saltos o memoria. Todos ellos tienen una estructura común: dado un conjunto de inputs, que pueden ser características del contrato y parámetros del modelo, devuelven un precio o una volatilidad implícita. El problema es que conforme los modelos ganan realismo también se vuelven más caros computacionalmente. Lo que en Black-Scholes es una fórmula cerrada se convierte en Heston en una integral que hay que resolver por Fourier, y en Bates o en *rough volatility* en algo aún más pesado. Cuando estos modelos hay que evaluarlos miles o millones de veces, por ejemplo en calibración diaria, en cálculo de Greeks o en simulación de escenarios, el coste se vuelve un cuello de botella real.

La idea de los *deep surrogates* es resolver ese cuello de botella entrenando una red neuronal a partir de datos sintéticos generados por el propio modelo, de forma que la red aprenda la función de pricing. Si llamamos $f : \mathcal{X} \subset \mathbb{R}^d \rightarrow \mathbb{R}$ a la función de pricing del modelo verdadero, lo que hacemos es entrenar una red $\hat{f}_\theta$ con parámetros θ para que

$$\hat{f}_\theta(x) \approx f(x), \quad x \in \mathcal{X}. \quad (1)$$

Una vez entrenada, evaluarla es órdenes de magnitud más rápido que llamar al solver original. El esquema general se ilustra en la [Figura 1](#).

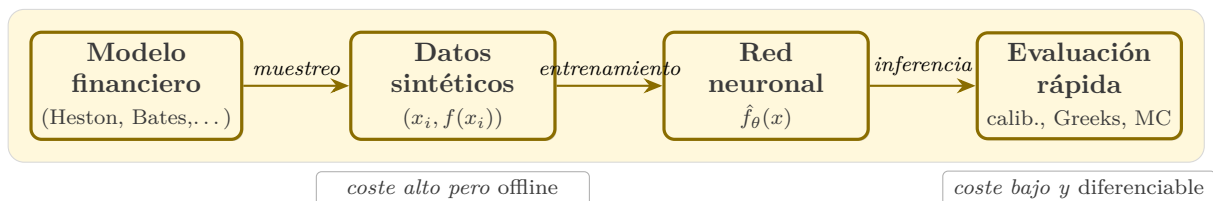


Figura 1: Esquema general de un *deep surrogate*: el modelo financiero genera datos sintéticos sobre los que se entrena una red que después sustituye al solver original en evaluaciones masivas.

La red no reemplaza la teoría financiera, simplemente aprende a imitar al solver. El objetivo no es aplicar las técnicas de *deep learning* a datos de mercado ruidosos, sino utilizar estas técnicas para resolver un problema clásico de aproximación numérica con la ventaja añadida de que el resultado es diferenciable por construcción.

En Black-Scholes la fórmula cerrada ya es muy rápida y un surrogate no aporta valor práctico, pero es ideal como caso de validación, porque conocemos la solución exacta y podemos medir errores de precio, Delta y volatilidad implícita con precisión. El objetivo del proyecto es verificar si se pueden utilizar surrogates con modelos más complejos sin perder precisión, y para ello combinamos Black-Scholes como entorno controlado con Heston como caso principal de estudio. La pregunta de investigación que estructura todo el trabajo se formula así:

Bajo qué condiciones una red neuronal profunda puede actuar como un surrogate preciso, rápido y diferenciable para funciones de pricing de opciones.

Para responderla de forma operativa, descomponemos la pregunta en cinco preguntas más concretas que se traducen en cinco experimentos comparativos. Cada uno varía una sola dimensión del problema mientras mantiene todo lo demás fijo, lo que permite atribuir cualquier diferencia observada a la dimensión que estamos variando y no a otras variables que se hayan colado por accidente.

El resto del documento se organiza como sigue. La [Sección 2](#) sintetiza la revisión bibliográfica que ha guiado las decisiones metodológicas, con especial detalle en la referencia central de Chen, Didisheim y Scheidegger. La [Sección 3](#) explica cómo enfocamos nuestro propio trabajo a partir de esa base. La [Sección 4](#) fija la metodología experimental: *targets*, normalización, solvers, dominio, muestreo y evaluación por *bins*. La [Sección 5](#) describe los cinco experimentos y los once surrogates que los sostienen. La [Sección 6](#) presenta el plan de ejecución por fases. La [Sección 7](#) discute alcance, limitaciones y trabajo en curso.

2. Revisión bibliográfica

La primera fase de nuestro trabajo ha consistido en una investigación bibliográfica profunda para tener una visión global de los temas relevantes y situar nuestra propuesta dentro del panorama existente. Hay cinco artículos que hemos trabajado a fondo y que forman el núcleo de la revisión: los tres primeros eran una recomendación de los profesores como punto de partida, un cuarto, el de *deep surrogates*, lo encontramos buscando trabajos más recientes en la línea de aproximación numérica de modelos de pricing, y un quinto, el de *differential machine learning* de Hugué y Savine, lo hemos incorporado como complemento al anterior porque ofrece una pieza metodológica que conecta directamente con nuestras preocupaciones sobre la calidad de las derivadas aprendidas.

2.1. Hutchinson, Lo y Poggio (1994): el punto de partida histórico

El punto de partida histórico es el trabajo de Hutchinson, Lo y Poggio [1]. Estos autores proponen un método no paramétrico para estimar la fórmula de pricing de un derivado mediante *learning networks*, en una época en la que la literatura financiera estaba dominada por enfoques paramétricos basados en Black-Scholes y argumentos de no arbitraje. La idea central es invertir la lógica habitual: en lugar de partir de un modelo estocástico para el subyacente y derivar analíticamente la fórmula del precio, dejan que la red aprenda directamente la función que mapea variables económicas observables, como el precio del subyacente normalizado por el strike y el tiempo a vencimiento, al precio de la opción, sin imponer supuestos de lognormalidad ni continuidad de trayectoria. Comparan cuatro arquitecturas, redes de funciones de base radial, perceptrones multicapa, *projection pursuit regression* y mínimos cuadrados ordinarios como baseline, y atacan a la vez los problemas de pricing y delta-hedging, lo cual es importante porque la cobertura exige que la red aproxime bien no solo el precio sino también su derivada respecto al subyacente. Validan primero con datos sintéticos generados por Monte Carlo bajo Black-Scholes y después con datos reales de opciones sobre futuros del S&P 500 entre 1987 y 1991, donde la red llega a superar a Black-Scholes en *tracking error* de la cartera replicante.

Su relevancia histórica reside en que es el primer trabajo que demuestra de forma sistemática que una red neuronal puede aprender una fórmula de pricing y usarse operativamente para cubrir, abriendo la línea de investigación que conecta con el enfoque actual de *deep surrogates*. La diferencia crucial con nuestro proyecto es que aquí la red aprende del mercado, mientras que un surrogate moderno aprende la salida de un pricer ya conocido para acelerarlo: la red pasa de competir con Black-Scholes a aproximarlos de manera eficiente.

2.2. Becker, Cheridito y Jentzen (2019): deep optimal stopping

El segundo paper es el de Becker, Cheridito y Jentzen sobre *deep optimal stopping* [2]. Aquí los autores proponen un método de *deep learning* para resolver problemas de parada óptima en tiempo discreto, es decir, problemas en los que hay que decidir en cada instante si parar o continuar un proceso de Markov para maximizar la esperanza de un pago. La idea central es que cualquier tiempo de parada admisible se puede descomponer en una sucesión de decisiones binarias, una por cada paso temporal, y que cada una de esas decisiones puede aprenderse como una función del estado actual mediante una red neuronal *feedforward*. Sustituyen la indicadora dura por una sigmoide, lo que permite entrenar los parámetros con ascenso de gradiente estocástico sobre trayectorias Monte Carlo, y proceden por inducción hacia atrás desde el último instante hasta el primero. El método se aplica al pricing de opciones bermudas tipo *max-call* sobre cestas de hasta quinientos activos en un Black-Scholes multidimensional, a un *callable multi barrier reverse convertible* y al problema no markoviano de parar óptimamente un movimiento browniano fraccionario, con precisión alta y tiempos cortos en dimensiones donde los métodos tradicionales sufren la maldición de la dimensionalidad.

Para nuestra revisión es importante porque ilustra una línea claramente distinta de la nuestra: ellos aprenden directamente la política de ejercicio para derivados con derecho de parada anticipada, mientras que en nuestro proyecto la red actuará como surrogate de la función de pricing de opciones europeas. Son enfoques complementarios dentro del uso del *deep learning* en finanzas cuantitativas, pero no resuelven el mismo problema, y conviene tener esa distinción clara para evitar comparaciones que no aplican.

2.3. Ruf y Wang (2020): mapa del campo

El tercer paper recomendado es la revisión bibliográfica de Ruf y Wang [3], que funciona como mapa general del campo. Los autores compilan y comparan más de ciento cincuenta artículos en una tabla detallada que cataloga cada estudio según seis dimensiones, entre ellas las variables de entrada, las salidas de la red, los modelos *benchmark*, las métricas de rendimiento, el método de partición de los datos y los activos subyacentes. Cubren tanto el enfoque puramente no paramétrico, donde la red aprende directamente el precio a partir de variables como subyacente, strike, vencimiento y volatilidad, como los enfoques híbridos que combinan fórmulas paramétricas tipo Black-Scholes con correcciones aprendidas, además de variantes más recientes orientadas a calibración, resolución de PDEs, aproximación de funciones de valor en problemas de control óptimo y *deep hedging*.

Lo más útil de este paper son las lecciones que extrae del repaso exhaustivo: la importancia de usar inputs estacionarios como la moneyness en lugar de precio y strike por separado, la necesidad de elegir *benchmarks* que no trivialicen la comparación y, sobre todo, el problema del *data leakage* que aparece cuando la partición train/test se hace de forma aleatoria sobre datos con estructura temporal, rompiendo el orden cronológico, contaminando el test e infraestimando sistemáticamente el error de generalización. Para nuestro trabajo es una referencia útil porque ofrece un panorama de la disciplina, permite situar nuestra propuesta dentro de las distintas líneas históricas y nos advierte de errores típicos que conviene evitar, aunque la mayor parte de la bibliografía que analiza versa sobre el uso de técnicas de *machine learning* directamente sobre

datos de mercado y no sobre el problema de aproximar un solver conocido.

2.4. Chen, Didisheim y Scheidegger (2025): referencia central

El cuarto paper, y el que ha terminado convenciendonos como referencia central, es el de Chen, Didisheim y Scheidegger sobre *deep surrogates* en finanzas [4]. Este artículo propone los *deep surrogates* como una clase de aproximadores numéricos para modelos estructurales costosos, sustituyendo la función original por una red neuronal profunda entrenada con muestras simuladas que toma exactamente los mismos inputs y devuelve la misma salida a un coste computacional drásticamente menor, además de proporcionar gradientes a un coste que no escala con el número de inputs. El paper aplica esta idea al modelo de Bates con saltos doble exponenciales, que tiene cinco variables de estado y ocho parámetros estructurales, resultando en un input de trece dimensiones, y entrena la red sobre mil millones de observaciones sintéticas para mapear ese input a la volatilidad implícita Black-Scholes asociada.

Una idea crucial del paper es que la maldición de la dimensionalidad queda mitigada por dos factores que se refuerzan entre sí. Por un lado, los datos de entrenamiento se generan de manera sintética y determinista a partir del propio modelo, sin ruido estadístico, lo que elimina la necesidad de cubrir el espacio de inputs con una densidad de muestras que permita promediar errores aleatorios. Por otro lado, la función de pricing en modelos afines como Heston o Bates es analíticamente suave, lo que controla el problema de aproximación y permite a una red profunda representar el mapa de inputs a precio con un número de parámetros que escala de forma manejable con la dimensión. Esta intuición se apoya en resultados teóricos: el trabajo clásico de Barron [11] sobre aproximación de funciones con norma de Barron acotada y, más recientemente, los de Grohs, Hornung, Jentzen y von Wurstemberger [12] demuestran formalmente cómo las redes neuronales profundas escapan a la maldición de la dimensionalidad en la aproximación de soluciones de la ecuación de Black-Scholes en alta dimensión. Juntas, estas dos propiedades justifican que un *deep surrogate* sea una herramienta de aproximación numérica plausible en problemas de dimensión moderada-alta, no solo un truco que funciona empíricamente.

El paper también argumenta de forma explícita por qué las redes neuronales son la familia de aproximadores adecuada frente a alternativas naturales como polinomios globales, *splines*, *sparse grids* o *Gaussian processes*. Los polinomios funcionan razonablemente en funciones suaves pero les cuesta capturar rasgos locales fuertes como *kinks*, y eso en opciones es un problema serio porque el *payoff* tiene un *kink* claro en el strike, con oscilaciones de Runge si se intenta una aproximación global. Los *splines* capturan bien rasgos locales por tramos pero escalan muy mal en alta dimensión. Las *sparse grids* mitigan parte del problema de las mallas cartesianas pero rinden peor en dominios irregulares, frecuentes en finanzas por las restricciones entre parámetros. Los *Gaussian processes* son elegantes pero su coste matricial $O(n^3)$ los hace inviables con millones de observaciones. Las redes neuronales son las únicas que cumplen los cuatro criterios de forma razonable, como resume la [Tabla 1](#).

Método	Alta dimensión	No linealidades	Dominio irregular	Escala n
Polinomios globales	—	—	~	+
<i>Splines</i>	—	+	~	~
<i>Sparse grids</i>	~	+	—	~
<i>Gaussian processes</i>	+	+	+	—
Redes neuronales	+	+	+	+

Tabla 1: Comparación cualitativa de aproximadores numéricos en los cuatro criterios relevantes para *surrogates* de pricing, en línea con el argumento de Chen et al. Las redes neuronales son las únicas que cumplen los cuatro de forma razonable.

A esta capacidad de aproximación se añade una ventaja decisiva en el contexto de los métodos

numéricos: el jacobiano de la red está disponible a un coste muy reducido mediante diferenciación automática. Esto es crítico en finanzas porque muchas magnitudes que importan no son el precio en sí sino sus derivadas, las *Greeks*, que en el modelo original suelen calcularse por diferencias finitas a un coste considerable. Chen et al. documentan que la calibración diaria del modelo de Bates pasa de las aproximadamente 125 horas que requiere la inversión FFT a poco más de dos horas en CPU y unas decenas de minutos en GPU cuando se usa el surrogate, una reducción de órdenes de magnitud que abre la puerta a aplicaciones empíricas hasta entonces inviables.

Cuatro decisiones técnicas concretas del paper son las que guían directamente las nuestras y se justifican en detalle en la [Sección 4](#) y la [Sección 5](#). Primero, usar una activación suave (Swish) en lugar de ReLU: ReLU produce errores de precio comparables pero Deltas sustancialmente peores porque su derivada es discontinua en cero, mientras que las activaciones suaves dan derivadas estables; esta observación motiva directamente nuestro experimento E2. Segundo, evaluar en volatilidad implícita Black-Scholes en vez de en precio, porque IV es la unidad común que pone todas las opciones en una escala homogénea sin importar moneyness ni vencimiento; nosotros mantenemos el precio como *target* de entrenamiento pero evaluamos en ambas escalas, y esa comparación es justamente E1. Tercero, definir un hipercubo amplio y muestrear puntos aleatorios dentro de él, recortando un cinco por ciento de los bordes en evaluación para no contaminar las métricas con errores de frontera; este esquema es la base de nuestro pipeline de generación de datos y E3 explora muestreos no uniformes dentro de él. Cuarto, prescindir de la regularización clásica del *machine learning* (sin *early stopping*, sin *dropout*, sin *weight decay*) porque los datos son sintéticos y deterministas, sin nada que sobreajustar.

La validación que hace el paper del aproximador es la otra pieza que copiamos. Separan explícitamente dos niveles de error: el error de aproximación, que mide cuánto se aleja el surrogate del solver original para parámetros conocidos y se reporta por *bins* de moneyness y vencimiento, y el error de estimación, que mide el coste económico de calibrar con el surrogate dentro de un procedimiento de optimización. En el primer nivel reportan que el percentil 95 del error absoluto en volatilidad implícita y en Delta queda por debajo de $8 \cdot 10^{-4}$ para opciones con más de siete días a vencimiento. Esta distinción entre los dos niveles de error es la que nos lleva a evaluar siempre por *bins* en lugar de reportar promedios globales y a separar precio, IV y Delta como métricas independientes en los cinco experimentos.

2.5. Huge y Savine (2020): differential machine learning

El quinto paper es el de Huge y Savine sobre *differential machine learning* [5]. Originalmente lo teníamos como uno más del panorama, pero al pensar en cómo podríamos innovar respecto al paper de *deep surrogates* nos hemos dado cuenta de que ofrece una pieza metodológica muy potente y muy alineada con nuestras preocupaciones. La idea es que cuando se entrena un surrogate con datos sintéticos, normalmente la red solo ve el valor de la función en cada punto, por ejemplo el precio de la opción, y aprende a predecirlo. Sin embargo, si el modelo verdadero es diferenciable, también se conoce la pendiente de la función en cada punto, es decir, las *Greeks*. *Differential machine learning* aprovecha esa información añadiendo los gradientes verdaderos al conjunto de entrenamiento y modificando la pérdida para que la red no solo estime el precio, sino también su derivada respecto a los inputs. El resultado es que se consiguen aproximaciones mucho más precisas, especialmente en términos de *Greeks*, con datasets sustancialmente más pequeños.

Para nuestro trabajo es una referencia clave por dos razones. La primera es que conecta directamente con la necesidad de validar *Greeks* que ya teníamos heredada del paper de Chen et al. La segunda es que tiene una motivación matemática sólida desde el punto de vista de aproximación de funciones, análoga a la que justifica la interpolación de Hermite frente a la clásica: disponer del valor y de la derivada de una función en cada punto muestreado permite reconstruirla con

muchos menos puntos. La Figura 2 ilustra esta intuición.

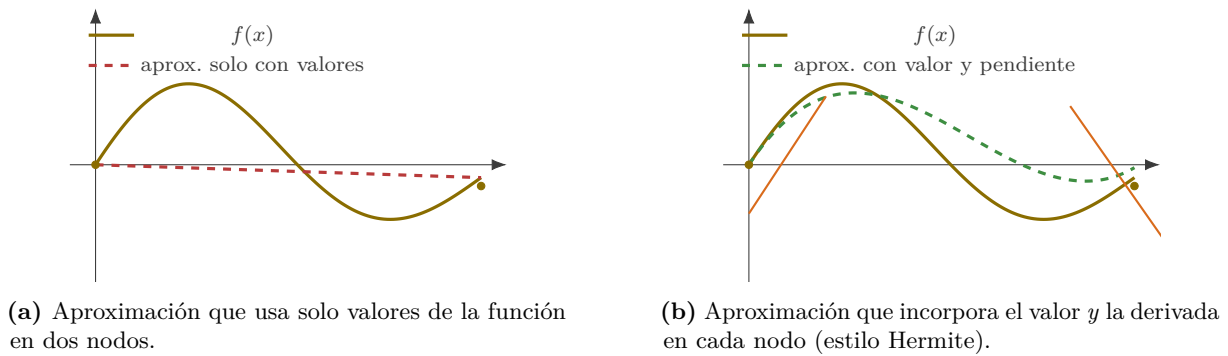


Figura 2: Motivación matemática del *differential machine learning*: añadir información sobre la derivada en cada nodo permite reconstruir la función con muchos menos puntos. Los segmentos naranjas representan las pendientes conocidas.

2.6. Otras líneas complementarias

Más allá de estos cinco pilares, hemos revisado otros trabajos que ayudan a entender que la aplicación de redes neuronales a derivados ha avanzado en varias direcciones complementarias. Una primera línea ataca de frente el problema de resolver las ecuaciones diferenciales parciales que aparecen en el pricing de derivados de alta dimensión, donde los métodos de mallado tradicionales chocan con la maldición de la dimensionalidad: en esa familia se inscribe el *Deep Galerkin Method* de Sirignano y Spiliopoulos [6], que entrena una red profunda para satisfacer el operador diferencial sobre puntos muestreados al azar y prescinde por completo de la malla, así como el *Deep BSDE Solver* de Han, Jentzen y E [7], que reformula la PDE como una ecuación diferencial estocástica retrógrada y aproxima el gradiente de la solución mediante redes neuronales, demostrando viabilidad incluso en problemas de cien dimensiones.

Una segunda línea busca acelerar la valoración aprendiendo directamente la función de precios, y aquí el trabajo de Ferguson y Green [8] muestra que una red profunda entrenada con datos sintéticos generados por Monte Carlo puede valorar opciones *basket* millones de veces más rápido que el modelo subyacente. Una tercera vertiente aborda la cobertura de carteras como un problema de aprendizaje por refuerzo, como en el *deep hedging* de Buehler et al. [9], que parametriza la estrategia de cobertura mediante redes neuronales y la optimiza directamente bajo medidas de riesgo convexas, incorporando de forma natural costes de transacción y otras fricciones de mercado sin necesidad de *Greeks*. La calibración de modelos estocásticos también se ha beneficiado de estas técnicas: trabajos recientes aplican *differential ML* al modelo de Heston para reducir drásticamente el tiempo de calibración [10].

Después de revisar todo este material, la conclusión a la que hemos llegado es que el paper de *deep surrogates* es el que mejor encaja con nuestros objetivos. La razón principal es que aborda un problema que en la práctica de la industria es muy importante, abaratar el coste computacional de los modelos de pricing y reducir los tiempos de calibración y estimación de parámetros, y son temas muy actuales en mesas de derivados y en *research* cuantitativo a medida que los modelos se vuelven más realistas y por tanto más caros de evaluar. Además, desde el punto de vista de métodos numéricos, el enfoque de *surrogates* es el más completo e interesante porque entrelaza muestreo, aproximación de funciones, validación numérica y cálculo de derivadas.

3. Enfoque del trabajo

Después de digerir esta bibliografía, la decisión más importante es cómo enfocar nuestro propio trabajo para complementar y aportar valor al material existente dentro de nuestras limitaciones temporales y computacionales. La estructura más coherente es trabajar en dos niveles. El primer nivel es Black-Scholes como caso de validación. Black-Scholes en la práctica no necesita un surrogate porque la fórmula cerrada ya es muy rápida, pero precisamente por eso es ideal como entorno de prueba: conocemos la solución exacta, conocemos las Deltas analíticas y podemos comparar precio, volatilidad implícita y *Greeks* de forma directa antes de complicar el problema. El segundo nivel es extender a Heston. Heston introduce volatilidad estocástica, es bastante más realista y su evaluación requiere métodos numéricos como Fourier semi-cerrado, así que la motivación de tener un surrogate empieza a cobrar sentido real. Es un paso intermedio entre Black-Scholes y Bates que conserva la esencia del paper de referencia sin obligarnos a implementar saltos doble exponenciales.

A partir de ahí, hemos identificado cinco puntos en los que centrar nuestro proyecto de investigación, que enumeramos aquí y desarrollamos en la [Sección 5](#) como experimentos formales.

El primero es comparar de forma sistemática el error en precio frente al error en volatilidad implícita. Planteamos como pregunta metodológica explícita hasta qué punto puede un surrogate tener un error en precio aparentemente bajo pero un error relevante en *implied volatility* en ciertas zonas del dominio. Esto acerca el trabajo a la práctica real del mercado, donde la volatilidad implícita es la unidad natural de comparación.

El segundo elemento diferencial es comparar funciones de activación con foco en la calidad de los *Greeks*. Entrenaremos redes con la misma arquitectura cambiando solo la activación, probando ReLU, *Softplus*, *Swish/SiLU* y *tanh*, y compararemos el MAE de precio, el MAE de Delta y el MAE de volatilidad implícita en cada caso. La hipótesis, alineada con Chen et al., es que las activaciones suaves producen derivadas más estables aunque los errores de precio sean parecidos. Mostrar que dos redes con MAE de precio similar pueden tener Deltas muy distintas es una conclusión potente para un trabajo de métodos numéricos.

El tercer elemento es comparar muestreo uniforme frente a muestreo enfocado. Generaremos dos *datasets*, uno con muestreo uniforme en todo el dominio y otro con más densidad cerca del *at-the-money* y de vencimientos cortos, donde la función de pricing tiene más curvatura. Evaluaremos ambos surrogates con la misma división por *bins* y analizaremos si el muestreo enfocado reduce el error en regiones críticas. Esto conecta directamente con ideas clásicas de métodos numéricos como la elección adaptativa de puntos de malla.

El cuarto elemento es la medición seria de eficiencia computacional. En Black-Scholes la ganancia será probablemente pequeña porque la fórmula cerrada ya es rapidísima, pero el experimento sirve para validar el pipeline. En Heston mediremos el tiempo necesario para valorar grandes volúmenes de opciones con el solver original frente al surrogate, en función del tamaño del lote, porque la red aprovecha mejor el paralelismo cuando procesa muchos puntos a la vez y reportar un único número aislado sería engañoso.

El quinto elemento, central en nuestro planteamiento, es estudiar el efecto de la pérdida en la calidad del surrogate con especial atención al enfoque de *differential machine learning* de Huge y Savine. En lugar de entrenar la red solo con precios, modificamos la pérdida para que también penalice la diferencia entre la Delta verdadera y la Delta que sale por *autograd* de la red. De este modo, la pérdida queda como una combinación ponderada del error de precio y del error de la derivada y la red aprende simultáneamente a acertar el nivel y la pendiente de la función. El planteamiento tiene una motivación matemática sólida ([Figura 2](#)), análoga a la que justifica la interpolación de Hermite frente a la clásica: disponer del valor y de la derivada en cada punto muestreado permite reconstruir la función con muchos menos puntos. La hipótesis asociada es

que el *differential machine learning* ofrece una ventaja sustancial frente al entrenamiento con pérdidas estándar en regímenes de *datasets* pequeños y que esa ventaja se atenúa conforme crece el tamaño de la muestra.

4. Metodología

Esta sección concreta cómo se traducen las preguntas anteriores en un pipeline experimental ejecutable. Su función no es añadir más tareas sino fijar cómo se ejecuta cada experimento, qué magnitudes se entrenan, qué métricas se reportan y bajo qué condiciones consideramos que una comparación es limpia. En un proyecto de *surrogates*, la parte delicada no es solo entrenar redes, sino asegurar que el dato sintético, la normalización, la métrica y la evaluación responden exactamente a la pregunta que se quiere contestar.

La idea central del proyecto es combinar dos líneas de la literatura. De Chen, Didisheim y Scheidegger [4] tomamos la lógica de *deep surrogates*: construir una red que aproxime un solver financiero caro a partir de datos sintéticos generados por el propio modelo. De Hufe y Savine [5] tomamos, de forma acotada, la idea de *differential machine learning*: si además del valor de la función conocemos una derivada económicamente relevante, podemos enseñar a la red la forma local de la función y no solo su nivel. En nuestro caso, esa derivada será la Delta.

4.1. Pipeline experimental

Todos los experimentos siguen el mismo flujo base. Primero se define un dominio de inputs y una distribución de muestreo. Después se genera un *dataset* sintético evaluando el solver verdadero en cada punto. A continuación se entrena un surrogate con una configuración cerrada y versionada. Finalmente se evalúa el surrogate sobre un conjunto independiente, separando los resultados por *bins* de moneyness y vencimiento. La separación entre generación, entrenamiento y evaluación es importante porque evita contaminar las conclusiones: el *sampler* decide dónde preguntamos al modelo verdadero, el solver decide cuáles son los *targets*, el *trainer* decide cómo aprende la red y el evaluador decide cómo medimos el error. La Figura 3 esquematiza la arquitectura concreta de la red que usamos y la relación entre el *target* de precio y la Delta obtenida por *autograd*.

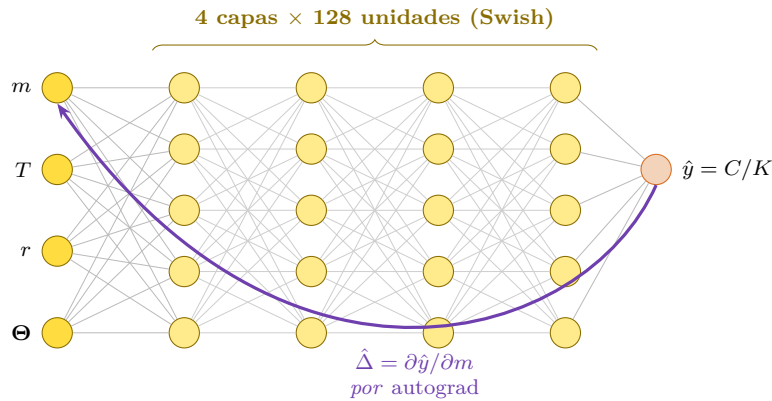


Figura 3: Arquitectura del surrogate: un MLP de cuatro capas ocultas y 128 unidades con activación Swish que toma como entrada la moneyness, el vencimiento, el tipo y los parámetros del modelo, y produce el precio normalizado $\hat{y} = C/K$. La Delta del surrogate se obtiene derivando $\hat{y}$ respecto a la moneyness por *autograd*, sin diferencias finitas.

4.2. Targets, normalización y métricas

El *target* principal de entrenamiento es el precio normalizado por strike, $y = C/K$, y el input financiero principal es la moneyness simple, $m = S/K$. Esta decisión elimina una escala redundante del problema: si *spot* y *strike* se multiplican por la misma constante, el precio de una *call* europea escala con el *strike*. Trabajar con C/K y S/K permite que la red aprenda una función más estable y facilita la interpretación de la Delta, porque

$$\Delta = \frac{\partial C}{\partial S} = \frac{\partial(C/K)}{\partial(S/K)} = \frac{\partial y}{\partial m}. \quad (2)$$

No usamos la moneyness normalizada de Chen et al. porque nuestro objetivo no es replicar la escala empírica del S&P 500, sino construir un entorno controlado donde la relación entre precio normalizado y Delta sea directa. También fijamos $q = 0$ en todos los experimentos: el *dividend yield* no forma parte de las preguntas centrales del trabajo y eliminarlo reduce la dimensión del dominio sin afectar a las comparaciones.

Además de esta normalización financiera, todos los inputs se normalizan numéricamente a $[0, 1]$ antes de entrar en la red. Esta normalización no cambia el problema matemático pero mejora la estabilidad del optimizador porque evita que dimensiones como T , ρ , v_0 o κ vivan en escalas muy distintas. Cuando se calculan derivadas de la red se aplica la regla de la cadena para volver a la escala financiera. En particular, si $m_{\text{norm}} = (m - m_{\text{mín}})/(m_{\text{máx}} - m_{\text{mín}})$, entonces

$$\hat{\Delta} = \frac{\partial \hat{y}}{\partial m} = \frac{1}{m_{\text{máx}} - m_{\text{mín}}} \cdot \frac{\partial \hat{y}}{\partial m_{\text{norm}}}. \quad (3)$$

La Delta del surrogate se calcula con *autograd* sobre la red, no con diferencias finitas. La Delta verdadera del *dataset* viene del solver ($N(d_1)$ en Black-Scholes y P_1 en Heston), mientras que la Delta predicha se obtiene derivando $\hat{y}$ respecto a m_{norm} y aplicando la corrección anterior. En evaluación se usa `create_graph=False` porque solo se mide la derivada. En el experimento que entrena con pérdida diferencial se usa `create_graph=True` porque la pérdida contiene $\text{MAE}(\Delta)$ y PyTorch necesita derivar esa pérdida respecto a los pesos de la red. Esto incrementa coste y memoria pero es el coste esperado del entrenamiento diferencial.

La volatilidad implícita Black-Scholes no será el *target* principal de entrenamiento, sino una métrica de evaluación. Esta elección mantiene el entrenamiento más directo, porque el solver genera precios, y evita introducir fallos de inversión de volatilidad implícita durante la generación del *dataset*. Al mismo tiempo, evaluar también en IV permite comprobar si un error pequeño en precio es realmente pequeño en la escala que se usa en mercado.

Las métricas mínimas de evaluación son $\text{MAE}(C/K)$, el error absoluto en precio normalizado, MAE_{IV} y MAE_{Δ} . Para cada una se reportan promedios y percentiles por *bin*, con especial atención al percentil 95, porque los errores de cola son los que revelan si el surrogate falla en regiones operativamente importantes.

4.3. Solvers y validación numérica

Black-Scholes funciona como entorno de control. El precio, la Delta, la Vega y la inversión a volatilidad implícita tienen fórmulas cerradas o procedimientos numéricos muy estables, así que cualquier fallo en Black-Scholes apunta a un error del *pipeline* y no a una dificultad intrínseca del modelo.

Heston es el caso principal. El precio se calcula mediante una formulación semi-cerrada de Fourier, expresada en términos de las probabilidades P_1 y P_2 ,

$$C = S e^{-qT} P_1 - K e^{-rT} P_2. \quad (4)$$

Esta formulación tiene una ventaja metodológica importante para el experimento de DML: la Delta de una *call* europea se obtiene como $\Delta = e^{-qT} P_1$. Si fijamos $q = 0$, entonces $\Delta = P_1$. Esta Delta es el *target* diferencial del experimento E5. Antes de generar *datasets* con Delta, la implementación se valida contra diferencias finitas centrales en una *grid* pequeña de puntos representativos del dominio. La validación no busca que las diferencias finitas sean el método principal, sino detectar errores de implementación, escalado o inestabilidad numérica. El criterio de aceptación es error absoluto medio inferior a 10^{-4} en Delta; los errores extremos se revisan manualmente porque pueden señalar problemas de integración, vencimientos demasiado cortos o puntos cercanos a frontera.

Más allá de los tests internos, la implementación se valida también de forma cruzada con QuantLib sobre una *grid* representativa de inputs. Esta validación externa cubre precio y Delta en Black-Scholes y en Heston, y es lo que cierra la primera fase del proyecto antes de generar *datasets* grandes. Mantener esta capa de validación es lo que permite descartar que cualquier patrón posterior se deba a un error silencioso en el solver, escenario que contaminaría todo el resto del trabajo.

La inversión a volatilidad implícita se usa solo en evaluación. El inversor debe controlar precios fuera de límites de arbitraje, zonas de Vega baja y fallos de convergencia. Los fallos no se ocultan: se reportan como parte del diagnóstico, especialmente en vencimientos muy cortos o regiones muy fuera del dinero.

4.4. Dominio y muestreo

Los *datasets* se generan muestreando un hipercubo de inputs. Esta decisión sigue la lógica de Chen et al.: tratamos los parámetros del modelo como pseudo-estados para que una única red aprenda la función de pricing en muchas parametrizaciones posibles. La distribución base es uniforme, porque cubre el dominio completo y no replica únicamente las regiones más frecuentes del mercado. La [Tabla 2](#) fija el dominio contractual común y el dominio adicional de Heston.

Variable	Rango	Uso
$m = S/K$	[0,4, 2,0]	BS y Heston
T	[7/365, 2,0]	BS y Heston
r	[0,00, 0,075]	BS y Heston
q	0	BS y Heston
σ	[0,03, 1,00]	solo BS
v_0	[0,0009, 1,00]	Heston, $\sqrt{v_0} \in [0,03, 1,00]$
θ	[0,0009, 1,00]	Heston, $\sqrt{\theta} \in [0,03, 1,00]$
κ	[0,10, 10,00]	Heston, reversión lenta a rápida
ξ	[0,10, 3,00]	Heston, vol-of-vol moderada a extrema
ρ	[-0,95, -0,05]	Heston, <i>skew</i> negativo típico de <i>equity</i>

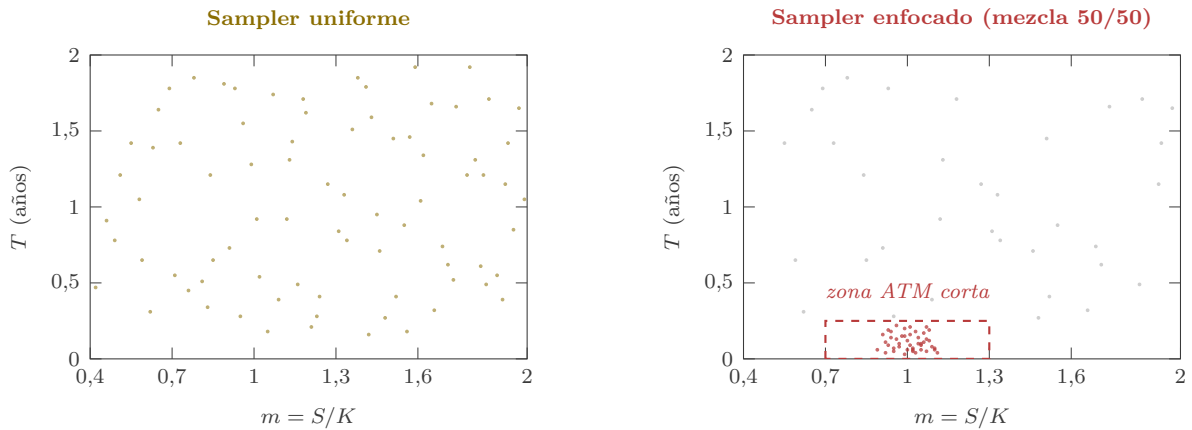
Tabla 2: Dominio contractual común y dominio adicional de Heston. Es deliberadamente amplio: incluye *weeklies*, *wings* profundas y volatilidades de estrés.

La elección de v_0 y θ se expresa en varianza pero se interpreta a través de su raíz cuadrada porque esa es la escala financiera natural. El rango cubre desde volatilidades bajas hasta episodios de estrés severo sin forzar al solver a vivir permanentemente en casos extremos. No imponemos la condición de Feller $2\kappa\theta \geq \xi^2$ como restricción de muestreo. Tratarla como filtro obligatorio haría el problema más limpio pero menos representativo de calibraciones reales, donde muchas violaciones aparecen en condiciones de mercado normales. En su lugar, se guarda como variable diagnóstica.

El muestreo *baseline* es uniforme en la escala financiera elegida. Para Black-Scholes, esto significa

uniforme directo sobre m , T , r y σ . Para Heston, $\sqrt{v_0}$ y $\sqrt{\theta}$ se muestrean uniformemente en $[0,03, 1,00]$ y después se elevan al cuadrado; κ , ξ y ρ se muestrean uniformemente en sus rangos. Esta decisión evita una distorsión importante: una uniforme directa sobre varianza sobreponderaría volatilidades muy altas.

La comparación de métodos de muestreo se concentra únicamente en el experimento E3. El resto de experimentos usa el *baseline* uniforme. En E3 se añade un *sampler* enfocado, con más masa cerca del *at-the-money* y de vencimientos cortos, donde la función de pricing tiene mayor curvatura. La construcción concreta es una mezcla 50/50: la mitad de las muestras sigue el *baseline* uniforme completo y la otra mitad usa $m \sim \text{TruncNormal}(1,0, 0,15, [0,7, 1,3])$ y $T \sim \text{LogUniform}(7/365, 0,25)$. La Figura 4 compara ambas distribuciones en el plano (m, T) .



(a) El muestreo uniforme cubre todo el hipercubo con densidad constante.

(b) El muestreo enfocado concentra masa en *at-the-money* y vencimientos cortos sin perder cobertura del resto.

Figura 4: Comparación del *sampler* uniforme (H-3) y del *sampler* enfocado (H-5). Ambos surrogates se evalúan sobre el mismo *test set* balanceado para que la comparación E3 mida dónde gana y dónde pierde el enfoque local.

4.5. Particiones y evaluación por bins

La partición de datos tiene tres niveles. El entrenamiento usa la distribución propia de cada surrogate. La validación usa una muestra independiente uniforme de cincuenta mil puntos por familia de modelo y sirve para monitorizar convergencia, activar el *scheduler* y seleccionar el *checkpoint*. La regla de selección es siempre el menor $\text{MAE}(C/K)$ en validación, incluso cuando la pérdida de entrenamiento incluye Delta. Mantener la regla fija es importante para que las comparaciones sean limpias: si el surrogate entrenado con DML se seleccionase con una métrica combinada de precio y Delta, no sabríamos si la mejora viene de la pérdida diferencial o de haber escogido otro punto de la trayectoria de entrenamiento.

El test final es independiente y balanceado por *bins*, y se comparte por todos los surrogates de una misma familia. Cada uno de los veinticinco *bins* tiene cinco mil puntos, para un total de ciento veinticinco mil observaciones. Esta decisión es especialmente importante en E3: si evaluásemos H-5 en una muestra con la misma concentración con la que entrena, confundiríamos precisión local con mejora global. El test balanceado permite ver exactamente dónde gana y dónde pierde el *sampler* enfocado.

La evaluación por *bins* es obligatoria porque los errores medios globales son insuficientes. Una red puede tener buen MAE agregado y, aun así, fallar sistemáticamente en opciones ATM de vencimiento corto o en regiones OTM donde la Vega es baja. Por eso usamos una partición 5×5 por moneyness y vencimiento, representada en la Figura 5. Las métricas se reportan en un

dominio interior que recorta un cinco por ciento de cada dimensión del hipercubo, para que los errores de frontera no dominen el promedio global.

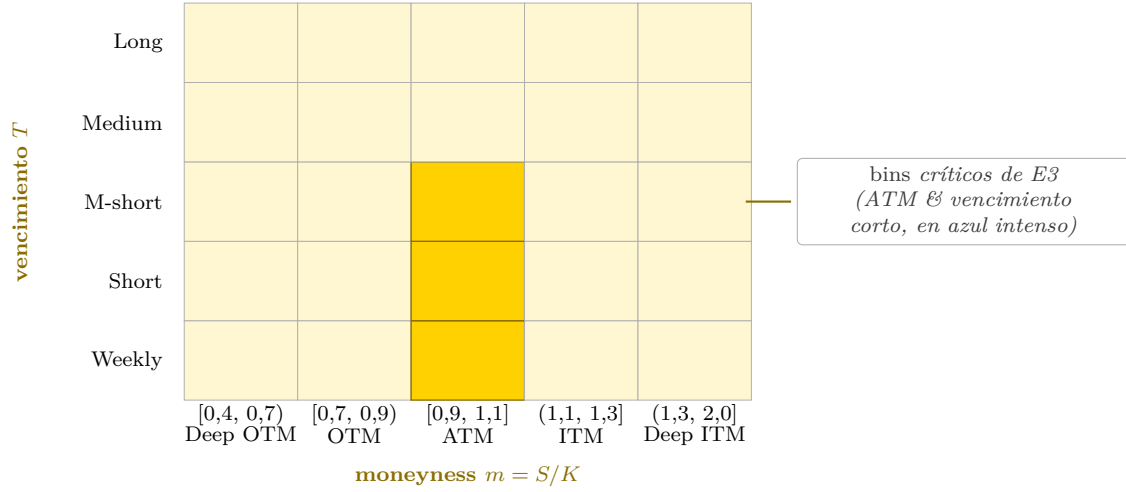


Figura 5: Partición 5×5 del dominio en *bins* de moneyness y vencimiento. Los *bins* ATM combinados con *weekly*, *short* y *medium-short* (sombreados en azul intenso) son los críticos para E3 por su mayor curvatura.

4.6. Constantes del pipeline

Para evitar reabrir discusiones durante la ejecución, fijamos los siguientes valores en código y no los movemos salvo que un hallazgo experimental lo justifique. Cada uno tiene una razón concreta que documentamos a continuación para que el porqué no se pierda con el tiempo.

Parámetro	Valor
Arquitectura	MLP, 4 capas ocultas, 128 unidades
Activación por defecto	Swish
Optimizador	Adam, $\text{lr} = 10^{-3}$, <i>scheduler</i> reduce-on-plateau
<i>Batch size</i>	1024 (<i>baseline</i>)
Épocas máximas	100
Regularización	ninguna (datos sintéticos)
Moneyness	$m = S/K$
Normalización de inputs	min-max a $[0, 1]$
<i>Output</i>	precio normalizado C/K
<i>Dividend yield</i>	$q = 0$
Recorte de bordes	5 % en cada cara del hipercubo
Semilla	fijada por configuración

Tabla 3: Constantes del *pipeline* comunes a todos los *surrogates*.

Un MLP *feed-forward* simple es suficiente porque el teorema universal de aproximación garantiza que basta para funciones suaves en dimensión moderada, y Chen et al. confirman empíricamente que funciona en Bates en dimensión trece. Cuatro capas ocultas y 128 unidades por capa dan del orden de cincuenta mil parámetros. Swish entra como activación por defecto porque, según Chen, es la decisión clave para que las Deltas obtenidas por *autograd* sean estables. Adam con $\text{lr} = 10^{-3}$ es el *default* de PyTorch y converge sin sintonización en la inmensa mayoría de problemas de regresión, y el *scheduler reduce-on-plateau* divide el *learning rate* por dos cuando la pérdida de validación se estanca.

La ausencia de regularización es la decisión metodológica más importante y viene directamente de Chen. En aprendizaje automático clásico la regularización existe porque los datos llevan

ruido y la red tendería a memorizarlo. En un surrogate, los *targets* vienen del propio solver, son deterministas y no hay nada que memorizar; la pérdida de validación nunca empeora por *overfitting*, solo se estanca cuando la red llega al límite de su capacidad expresiva. Por eso no usamos *early stopping*, *dropout* ni *weight decay*. El tope de cien épocas funciona como límite práctico.

5. Diseño experimental

Cinco experimentos cubren los cinco elementos diferenciales descritos en la [Sección 3](#). La elección de exactamente cinco no es arbitraria: es la lectura directa del bloque de elementos diferenciales, traducido a comparaciones experimentales concretas. Cada experimento responde a una pregunta única, varía una sola dimensión y mantiene todo lo demás fijo. Esta rigidez es lo que permite atribuir cualquier diferencia observada a la dimensión que estamos variando y no a otras variables que se nos hayan colado por accidente.

ID	Pregunta	Lo que varía	Surrogates	Resultado esperado
E1	¿Un error bajo en precio implica un error bajo en IV?	Métrica de evaluación	BS-3, H-3	<i>Bins</i> donde el MAE de precio es bajo pero el de IV no lo es.
E2	¿Qué activación produce mejores <i>Greeks</i> sin degradar el precio?	Activación	BS-1..4, H-1..4	MAE de precio similar, MAE de Delta mayor en ReLU.
E3	¿El muestreo enfocado reduce el error en <i>bins</i> críticos?	Distribución de muestreo	H-3 vs H-5	H-5 mejor en ATM y vencimiento corto, posiblemente peor en <i>wings</i> .
E4	¿Qué <i>speedup</i> ofrece el surrogate vs el solver?	Tamaño de lote	H-3	<i>Speedup</i> creciente con el lote, especialmente en GPU.
E5	¿Entrenar con Delta diferencial mejora la eficiencia muestral?	<i>Targets</i> de entrenamiento	H-3-small, H-6-small, H-3	H-6-small con mejor Delta y precio comparable a H-3-small.

Tabla 4: Los cinco experimentos del trabajo. Cada uno responde a una pregunta única, varía una sola dimensión y mantiene todo lo demás fijo.

E1 es observacional. No requiere entrenar nada nuevo, simplemente recalcula métricas sobre un surrogate que ya está entrenado para otros experimentos. La razón por la que merece llamarse experimento aparte es que la pregunta que responde es metodológica: nos dice si dos surrogates con MAE de precio aparentemente similares pueden tener errores muy distintos en la métrica que de verdad importa en el mercado, que es la volatilidad implícita.

E2 es el experimento más cargado en surrogates porque la activación es la única decisión arquitectónica que afecta directamente la calidad de las derivadas. La intuición se ve en la [Figura 6](#): ReLU, *Softplus*, Swish y *tanh* producen perfiles de salida parecidos, pero solo ReLU tiene una derivada discontinua en cero, mientras que las otras tres son suaves en todo el dominio. Cuando la Delta del surrogate se obtiene por *autograd*, esa discontinuidad de la activación se propaga al gradiente y degrada la calidad de la Delta aprendida, aunque el error en precio sea comparable. Entrenar las cuatro activaciones sobre Black-Scholes nos sirve como control en un entorno donde conocemos la solución exacta y las Deltas analíticas, y entrenarlas sobre Heston confirma o desmiente el patrón en el caso real.

E3 ataca una idea clásica de métodos numéricos transferida al dominio de los *surrogates*: concentrar el esfuerzo computacional en las regiones donde la función es más difícil de aproximar.

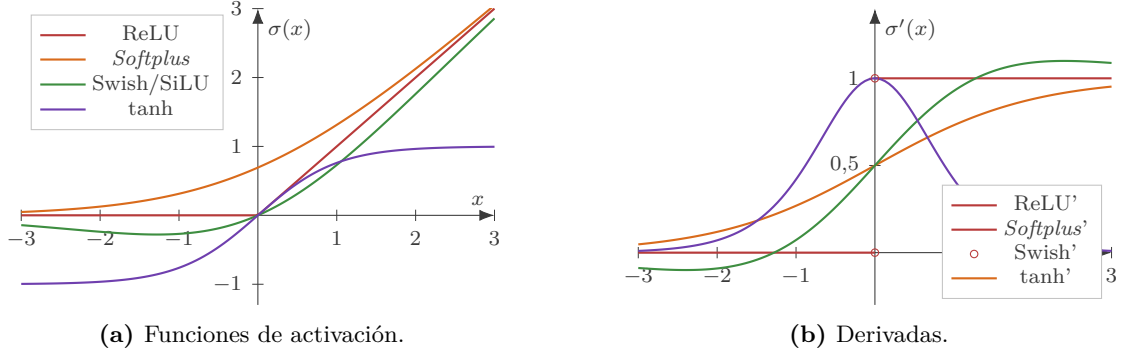


Figura 6: Las cuatro activaciones del experimento E2 y sus derivadas. La derivada de ReLU es discontinua en cero, lo que se propaga a las Deltas calculadas por *autograd*; las otras tres son suaves en todo el dominio.

La comparación es limpia porque H-3 y H-5 difieren solo en la distribución de muestreo: misma arquitectura, mismo número total de puntos, misma pérdida y mismo tamaño total del *dataset*.

E4 es de medición, no de comparación. Cronometra evaluaciones de H-3 contra el solver original de Heston en lotes de tamaño creciente. Reportar el *speedup* como función del tamaño de lote es más honesto que dar un único número que dependería arbitrariamente del lote elegido.

E5 prueba la hipótesis central del paper de Hugel y Savine en nuestro *setting* de forma acotada: si además de precios generamos Delta verdadera, ¿puede el surrogate aprender mejor la forma local de la función de pricing con menos datos? Delta es la *Greek* más limpia para este primer experimento porque, usando $y = C/K$ y $m = S/K$, se cumple $\Delta = \partial y / \partial m$.

Cada experimento tiene una métrica primaria alineada con su pregunta y métricas secundarias como control. Esto evita que el resultado se reduzca a una tabla grande sin conclusión clara.

ID	Métrica primaria	Controles y diagnóstico
E1	Discrepancia $MAE(C/K)$ vs MAE_{IV} por <i>bin</i>	Sin umbral; Vega/proxy, percentiles altos y fallos de IV
E2	MAE_{Δ} por <i>bin</i>	Sin umbral; activaciones suaves vs ReLU y precio como control
E3	MAE_{IV} en <i>bins</i> ATM con weekly, short, medium-short	Positivo fuerte si mejora $\geq 10\%$ y global empeora $\leq 10\%$; precio como control
E4	$speedup = t_{solver}/t_{surrogate}$ por tamaño de lote	Lotes 10^2-10^5 , 3 <i>warmups</i> + 10 repeticiones, mediana, CPU/GPU separado
E5	Mejora de MAE_{Δ} de H-6-small vs H-3-small	Positivo fuerte si Δ mejora $\geq 20\%$ y precio empeora $\leq 10\%$; distancia frente a H-3

Tabla 5: Métrica primaria y controles para cada experimento.

5.1. Once surrogates

Once *surrogates* en total. La cifra sale de trabajar hacia atrás desde los cinco experimentos: cada uno necesita un conjunto mínimo de *surrogates* entrenados para sostener su comparación y muchos se solapan. H-3, el *surrogate* de Heston con configuración *baseline*, aparece como referencia en los cinco experimentos. Esa centralidad evita que el coste computacional total se dispare y justifica que dediquemos especial cuidado a su entrenamiento.

En E5 sí varía deliberadamente el tamaño del *dataset* porque la pregunta no es solo si DML mejora las *Greeks*, sino si permite alcanzar precisión comparable con menos muestras. Si dejásemos

ID	Modelo	Activación	Pérdida	Dataset	Experimentos
BS-1	Black-Scholes	ReLU	$L_1(\text{precio})$	200k uniforme	E2
BS-2	Black-Scholes	Softplus	$L_1(\text{precio})$	200k uniforme	E2
BS-3	Black-Scholes	Swish	$L_1(\text{precio})$	200k uniforme	E1, E2
BS-4	Black-Scholes	tanh	$L_1(\text{precio})$	200k uniforme	E2
H-1	Heston	ReLU	$L_1(\text{precio})$	500k uniforme	E2
H-2	Heston	Softplus	$L_1(\text{precio})$	500k uniforme	E2
H-3	Heston	Swish	$L_1(\text{precio})$	500k uniforme	E1–E5
H-4	Heston	tanh	$L_1(\text{precio})$	500k uniforme	E2
H-5	Heston	Swish	$L_1(\text{precio})$	500k enfocado	E3
H-3-small	Heston	Swish	$L_1(\text{precio})$	100k uniforme	E5
H-6-small	Heston	Swish	$L_1(\text{precio}) + L_1(\Delta)$	100k uniforme + Δ	E5

Tabla 6: Los once *surrogates* del trabajo. Todos los comparados dentro de un mismo experimento comparten arquitectura, optimizador, hiperparámetros y semilla.

variar la arquitectura entre *surrogates*, la comparación de activaciones quedaría contaminada con el efecto del cambio arquitectónico.

5.2. Criterios de validez

Cada experimento se cierra con una comparación que cambia una sola dimensión relevante. Si se cambia activación, no se cambia *dataset*. Si se cambia *sampler*, no se cambia arquitectura. Si se cambia *loss* en E5, se mantiene el mismo tamaño pequeño de *dataset* para H-3-small y H-6-small. Para E3 y E5, en los que sí buscamos una clasificación operativa del resultado, los umbrales *positivo fuerte/débil/negativo* se fijan antes de ver los datos. Esa regla previa es lo que evita interpretar resultados de forma oportunista cuando llegue el momento de redactar conclusiones.

6. Plan de ejecución por fases

El proyecto se divide en cuatro fases ordenadas. Cada una no empieza hasta que la anterior está cerrada y verificada contra un criterio de salida concreto, como ilustra la **Figura 7**. Esta secuencialidad es estricta porque las fases tienen dependencias reales: no podemos entrenar redes sobre datos que no hemos generado, no podemos generar datos sin *solvers* que funcionen y no podemos validar un experimento sin métricas implementadas.

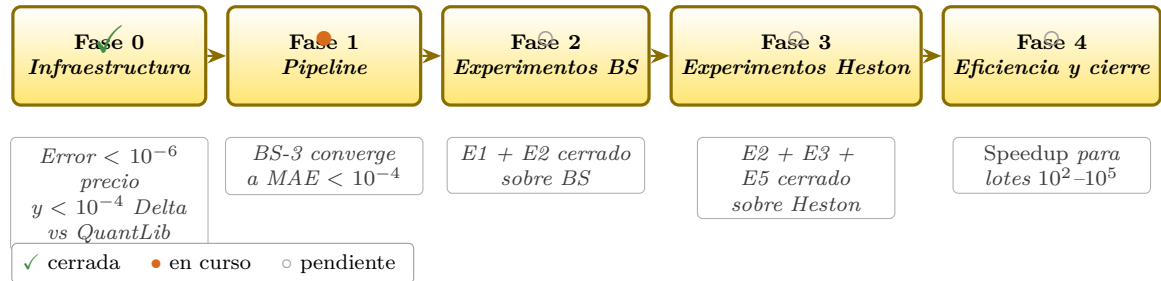


Figura 7: Plan de ejecución secuencial por fases con su criterio de salida. La secuencialidad es estricta: cada fase depende del cierre verificado de la anterior.

La **fase 0** cubre la infraestructura. Incluye la estructura del repositorio, semillas centralizadas, *solvers* de Black-Scholes y Heston, inversor de volatilidad implícita y la validación cruzada con QuantLib. Su criterio de salida es que Black-Scholes y Heston reproduzcan valores de referencia externos con error inferior a 10^{-6} en precio y 10^{-4} en Delta. Esta fase se cierra contra valores

tabulados conocidos porque un fallo silencioso en el *solver* contaminaría todo el resto del proyecto sin que nos enteremos.

La **fase 1** monta el *pipeline* de datos y entrenamiento. Implementa el *sampler*, el generador, el *script* reproducible de generación, el modelo, las utilidades de *Greeks*, el *trainer* y el *script* de entrenamiento. Genera los *datasets* base de entrenamiento, validación y test balanceado por *bins* para ambas familias de modelo. Su criterio de salida es un *smoke test* sobre BS-3 que converja a un MAE de precio razonablemente pequeño en validación. Es la primera vez que el *pipeline* completo se ejecuta de principio a fin: si BS-3 no converge cuando le pasamos datos de una fórmula cerrada conocida, hay algo roto antes de tocar Heston y este es el momento de descubrirlo.

La **fase 2** ejecuta los experimentos sobre Black-Scholes. Entrena BS-1, BS-2, BS-3 y BS-4, implementa los módulos de *binning* y métricas, produce tablas y *heatmaps* por *bin* para cada *surrogate* y completa los análisis E1 (precio vs IV) y E2 (activaciones) sobre Black-Scholes. El motivo de hacer Black-Scholes antes que Heston es que Black-Scholes tiene fórmulas cerradas para precio, Delta, Vega e IV, lo que permite validar todas las piezas del *pipeline* contra valores exactos antes de extender el mismo análisis a Heston.

La **fase 3** ejecuta los experimentos sobre Heston, que es el bloque más grande del proyecto. Genera los *datasets* específicos de E3 y E5, entrena H-1, H-2, H-3, H-4, H-5, H-3-small y H-6-small, repite el análisis E2 sobre Heston y cierra E3 (muestreo) y E5 (DML) con su clasificación operativa fuerte/débil/negativa.

La **fase 4** cubre eficiencia y cierre. Implementa el protocolo de cronometraje con *warmup* y diez repeticiones por lote, ejecuta E4 sobre H-3, produce las figuras finales y redacta la memoria. La eficiencia se mide al final porque depende de tener el *surrogate* ya entrenado y validado.

7. Discusión y trabajo futuro

El alcance del proyecto está deliberadamente acotado. No intentamos calibrar a datos reales de mercado, ni estudiar opciones exóticas, ni construir un *surrogate* industrial a escala de Chen et al. El objetivo es demostrar, en un entorno controlado y reproducible, cuándo una red puede aproximar un *solver* de pricing con precisión, rapidez y derivadas útiles. La contribución está en la comparación ordenada de métricas, activaciones, muestreo, eficiencia y aprendizaje diferencial, no en maximizar complejidad del modelo a cualquier coste.

Quedan abiertas varias extensiones naturales que merecen mención. Extender el conjunto de *Greeks* incorporadas a la pérdida diferencial (Vega, Gamma, Theta, Rho) permitiría medir hasta qué punto el *differential machine learning* escala más allá del caso unidimensional que estudiamos en E5, aunque introduce dificultades numéricas adicionales porque algunas de esas derivadas son sensibles a la inestabilidad del *solver* de Heston en zonas extremas. Pasar de opciones europeas a opciones americanas o exóticas pondría a prueba la generalización del enfoque a *payoffs* con derecho de ejercicio anticipado y conectaría nuestro trabajo con la línea de Becker, Cheridito y Jentzen. Aplicar el *surrogate* dentro de un procedimiento de calibración a datos reales — en línea con el segundo nivel de validación de Chen et al. — mediría el coste económico del error de aproximación de la red. Por último, escalar la arquitectura o el tamaño del *dataset* permitiría explorar si la reducción de error es lineal o se satura, una pregunta especialmente relevante en la fase actual del proyecto.

A día de hoy, las fases 0 y 1 están prácticamente cerradas: los *solvers* están validados contra QuantLib y el *pipeline* de generación y entrenamiento está operativo. Las fases 2 a 4 se ejecutarán en las semanas siguientes según el plan descrito en la [Sección 6](#). La estructura del trabajo está diseñada para que los resultados experimentales se puedan integrar de forma incremental en la

versión final del paper sin reescribir las secciones metodológicas, que ya están cerradas.

Referencias

- [1] J. M. Hutchinson, A. W. Lo y T. Poggio. A nonparametric approach to pricing and hedging derivative securities via learning networks. *Journal of Finance*, 49(3):851–889, 1994.
- [2] S. Becker, P. Cheridito y A. Jentzen. Deep optimal stopping. *Journal of Machine Learning Research*, 20:1–25, 2019.
- [3] J. Ruf y W. Wang. Neural networks for option pricing and hedging: a literature review. *Journal of Computational Finance*, 24(1):1–46, 2020.
- [4] H. Chen, A. Didisheim y S. Scheidegger. Deep surrogates for finance: with an application to option pricing. *Manuscript / SSRN preprint*, 2025.
- [5] B. Huge y A. Savine. Differential machine learning. *Risk Magazine* y SSRN, 2020.
- [6] J. Sirignano y K. Spiliopoulos. DGM: a deep learning algorithm for solving partial differential equations. *Journal of Computational Physics*, 375:1339–1364, 2018.
- [7] J. Han, A. Jentzen y W. E. Solving high-dimensional partial differential equations using deep learning. *Proceedings of the National Academy of Sciences*, 115(34):8505–8510, 2018.
- [8] R. Ferguson y A. Green. Deeply learning derivatives. *arXiv preprint arXiv:1809.02233*, 2018.
- [9] H. Buehler, L. Gonon, J. Teichmann y B. Wood. Deep hedging. *Quantitative Finance*, 19(8):1271–1291, 2019.
- [10] *Calibrating stochastic volatility models with differential machine learning. SSRN preprint*, 2023.
- [11] A. R. Barron. Universal approximation bounds for superpositions of a sigmoidal function. *IEEE Transactions on Information Theory*, 39(3):930–945, 1993.
- [12] P. Grohs, F. Hornung, A. Jentzen y P. von Wurstemberger. A proof that artificial neural networks overcome the curse of dimensionality in the numerical approximation of Black-Scholes partial differential equations. *arXiv preprint arXiv:1809.02362*, 2018.